UNIVERSIDAD
NEBRIJA

MASTER IN QUANTUM COMPUTING

Quantum Error Correction Decoders for 2D Color Codes:

Systematic Simulation and Comparison

Antonio Morales García

Scientific Director: Dr. Miguel Ángel Martín-Delgado Alcántara
Tutor: Prof. Francisco Gálvez Ramírez

Academic Year 2025/2026

Pendiente: dedicatoria a las personas
a las que más quiero, las que más me quieren.

Acknowledgments

[Pendiente] Agradecimientos a todos los que han hecho posible que haga este TFM.

Nota para no olvidar - Mis padres, que me inculcaron la curiosidad, la capacidad de cuestionar lo establecido y la perseverancia - Mis abuelos, que me inculcaron el amor propio (hacer las cosas bien) y me enseñaron a valorar el conocimiento y la sabiduría - Mi mujer y mis hijos, que han sacrificado casi tanto como yo para poder hacer esto (hacer un TFM pasados los 40 requiere grandes sacrificios familaires) - Mis mentores y profesores, que han ido construyendo el conocimiento que me ha permitido llegar a donde he llegado, y que me han inspirado para adentrarme en el mundo de la computación cuántica – los profesores son arquitectos invisibles de nuestra identidad - Y, el último y más importante, Miguel Ángel, por este privilegio

Contents

CHAPTER 1

Introduction: Classical error correction and foundations of quantum error correction

The field of quantum computing is evolving rapidly. It has transitioned from a theoretical curiosity to being generally accepted as a frontier technology. The fundamental premise of this discipline is that information processing based on the laws of quantum mechanics can solve problems that remain computationally out of reach for classical computing [1]. This paradigm began in 1982, when Richard Feynman pointed out that classical computers are not well equipped to simulate quantum systems due to the exponential growth of space required to represent a quantum state. He proposed that a computer based on quantum mechanical principles would be required to model nature effectively [2].

The Quantum Advantage and the Noise Barrier

The true potential of this technology was demonstrated in 1994 by Peter Shor, who developed a quantum algorithm for integer factorization in polynomial time [3]. Shor's work proved that quantum computing could break widely used cryptographic schemes, such as RSA. That is a theoretical proof of what has been called "Quantum Advantage". However, this assumes the existence of quantum hardware that has not been realized yet.

The realization of such a computer is difficult because of the fragility of quantum states. Qubits are vulnerable to noise from environmental interactions and to

inaccuracies in the operations we make with them. When this noise affects the qubits, the information they store is lost.

We refer to this reality as the era of Noisy Intermediate-Scale Quantum (NISQ) technology. In this era, error rates in quantum gates are several orders of magnitude higher than their classical counterparts. Without a robust way to protect quantum information, even the most advanced algorithms fail before reaching a meaningful result.

The Evolution of Quantum Error Correction

As we will see later in this chapter, unlike classical bits, quantum information cannot be protected by just copying it, due to the no-cloning theorem. To overcome this, the field of Quantum Error Correction (QEC) was pioneered by Peter Shor [4] and Andrew Steane [5]. They demonstrated that a single "logical" qubit can be represented with multiple "physical" qubits. More formally, the logical qubit is encoded in a larger Hilbert space of multiple physical qubits, allowing for the detection and correction of errors without destroying the underlying quantum state.

The Threshold Theorem [6] states that arbitrarily long quantum computation is only possible if the physical error rate is strictly below a certain critical value, known as the threshold.

As we will see in the following chapters, while Shor's code proves that QEC can be theoretically implemented, it is unable to effectively mitigate error, as the added complexity of the code causes more errors than it can prevent. This occurs when the physical error rate is above the fault-tolerance threshold, a regime known as "operating above the threshold". The fault-tolerance threshold depends on the specific QEC protocol, including the mechanism to encode the information and detect errors (known as **syndrome extraction**) and the algorithm used to infer the corrections (known as **the decoder**). On the other hand, the physical error rate depends on the specific hardware used.

A significant milestone in QEC was the introduction of topological codes by Alexei Kitaev [7], which will be discussed in Chapter 2. Kitaev's Toric Code and subsequent Surface Codes presented a new paradigm of local, lattice-based architectures where errors are detected by measuring local parities (stabilizers). These types of codes became leading candidates for scalable hardware due to their high error thresholds and two-dimensional physical layouts, which map well to the most common realizations of quantum computers.

Project Scope

This Master's Thesis focuses on the simulation and comparative analysis of decoders for the 2D triangular color code. Introduced by Bombín and Martin-Delgado [8], color codes represent an evolution over surface codes that offers significant advantages, particularly regarding the transversal implementation of logical gates, which simplifies fault-tolerant operations.

The scope of this work is defined by three specific constraints regarding the lattice geometry, the decoding algorithms, and the noise model. The foundations of these concepts are detailed in the following sections and chapters, starting

with an overview of classical error correction principles. Below, we outline the specific constraints.

First, as we will formally define in Chapter 2, color codes can be implemented through several geometries (4.8.8, 6.6.6 and 4.6.12). In this work, we will specifically adopt a **4.8.8 lattice geometry**. This choice is motivated by its lower qubit requirements (fewer qubits are needed per distance) and its properties as a doubly-even code, which enables transversal implementation of the full Clifford group, including H, $S^\dagger$ and CNOT [8].

As this introduction establishes the boundaries of the research, it references advanced concepts (like topological geometries or noise models) that will be formally defined in subsequent chapters.

Second, our objective is to evaluate the performance of various decoding strategies, from classical **Projection decoders** to state-of-the-art **Möbius** and **Concatenated MWPM** approaches. Using advanced simulation and sampling tools such as `stim` [9] and `sinter`, in Chapter 7 we will quantify their efficiency through standard metrics: physical error thresholds (p_{th}), logical failure rate scaling (p_L) as a function of physical noise, theoretical sub-threshold suppression limits, and the trade-off between decoding accuracy and computational time.

Finally, as we will discuss in Chapter 4, various sources of noise affect real quantum hardware. While realistic hardware-aware simulations of QEC codes require accounting for all of them—including imperfect gates and measurement errors—this study focuses strictly on **code-capacity noise**. This foundational model isolates the decoherence of data qubits over time by assuming perfect syndrome extraction. Adopting this approach allows us to evaluate the strengths of the 4.8.8 lattice and the algorithmic efficiency of the decoders, without the confounding variables introduced by circuit-level operations.

1.1 Classical error correction

Before discussing how to protect quantum information, it is essential to understand the basic mechanisms of classical error correction. While classical bits are physically robust, they are still susceptible to occasional errors (e.g. caused by hardware degradation or electromagnetic interference). Since classical systems are naively discrete (information is binary), a classical error is strictly discrete: a bit flips from 0 to 1, or from 1 to 0.

To detect and correct these bit-flips, classical computing relies on the concept of **redundancy**. By adding extra bits to the original message, we can deduce if an error has occurred and, in many cases, revert it. Two of the most foundational techniques used to achieve this are repetition codes and parity checks. These classical concepts serve as the baseline for building analogous quantum error correction solutions.

Repetition Codes and Parity Checks

The simplest approach to redundancy is the **repetition code**. If we want to send a single logical bit, we copy it multiple times. For example, in a 3-bit repetition code, a logical 0 is encoded as 000, and a logical 1 as 111.

For the 3-bit repetition code to be useful, the probability of a single bit flipping, p, must be low enough that the chance of two or three bits flipping simultaneously ($3p^2(1-p)+p^3$) is negligible.

If a noise event flips one of the physical bits (e.g., the state becomes 010), the receiver can apply a **majority vote** to determine the original message. Since there are more zeros than ones, the system concludes the intended logical state was 0, effectively correcting the error.

Figure 1.1. *Mechanism of a classical 3-bit repetition code correcting a single bit-flip error.*

This overlapping block parity concept is the foundation of classical Hamming codes, which can mathematically pinpoint exactly which bit flipped without needing massive repetition.

A more efficient method is the use of **parity checks**. Instead of copying the entire message, we calculate a parity bit based on the sum of the data bits. For instance, we can add a bit to ensure that the total number of 1s in a string is always an even number. If a single bit flips during transmission, the overall parity becomes odd, immediately alerting the system to the presence of an error. By structuring multiple parity checks across different overlapping blocks of data, classical systems can locate and correct errors with minimal overhead.

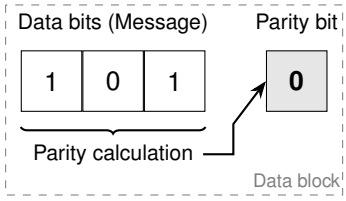


Figure 1.2. *A parity check bit appended to a 3-bit message. The parity bit (0) is calculated to ensure that the total number of '1's in the data block is even.*

The Three Major Challenges of Quantum Error Correction

It is natural to assume we could simply apply these classical techniques to quantum computers. However, translating repetition and parity concepts to the quantum realm is not straightforward. Quantum mechanics imposes three fundamental constraints that classical computing does not face:

1. **The No-Cloning Theorem:** Classical repetition codes require copying the information. In quantum mechanics, it is fundamentally impossible to create an identical copy of an unknown arbitrary quantum state [10]. Therefore, we cannot simply duplicate a qubit to create redundancy.
2. **Measurement Destroys Information:** Classical parity checks require reading the bits to see if an error occurred. If we attempt to measure a qubit in a superposition state to check for errors, the superposition collapses into a definitive classical state. Measuring the data to find the error destroys the quantum information we are trying to protect.
3. **Continuous Errors:** Classical computers are binary (a bit is either 0 or 1) and discrete (there are no intermediary values). This property applies

to classical errors (a bit is either right or wrong). Qubits, however, exist in a continuous state space, the complex Hilbert space. This means that there is an infinite number of possible errors that can occur, making it apparently impossible to design a code that accounts for every potential continuous deviation.

A qubit state, $|\psi\rangle$, is a vector in a two-dimensional complex Hilbert space, defined by continuous amplitudes $\alpha|0\rangle + \beta|1\rangle$, with α and β being complex numbers. Such state can be represented in the Bloch Sphere. Because of this continuity, an error can be an arbitrarily small rotation.

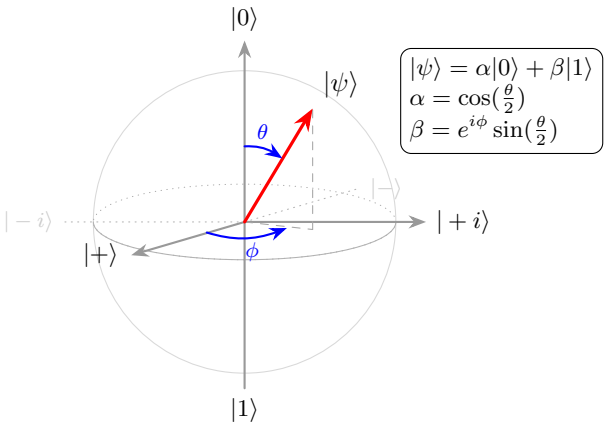


Figure 1.3. *Geometric representation of a qubit on the Bloch sphere. The parameters θ and ϕ define the continuous orientation of the state vector $|\psi\rangle$ relative to the computational basis states $|0\rangle$ and $|1\rangle$.*

The solution to these three fundamental barriers was formulated by Peter Shor, leading to the creation of the first true quantum error correction code, which we explore in the next section.

1.2 Shor codes

The transition from classical to quantum error correction was made possible by Peter Shor's 1995 proposal [4]. His approach demonstrated that the three barriers of quantum mechanics mentioned above could be bypassed using entanglement, parity-based measurements and concatenation of QEC codes.

The 3-qubit Code: Solving Cloning and Measurement

The first step is understanding that while we cannot copy an unknown state $|\psi\rangle$, we can **distribute** its information across an entangled system. In the 3-qubit bit-flip code, the encoding does not create copies; it creates a multi-particle state where the logical information is stored in a correlation of these particles' states.

$$|0\rangle_L = |000\rangle, \quad |1\rangle_L = |111\rangle$$

The 3-qubit codes exposed here, which are simple repetition codes, are the main components of the 9-qubit Shor code. It is important to note that in Shor's original paper [4] these 3-qubit codes were not introduced as independent, standalone protocols.

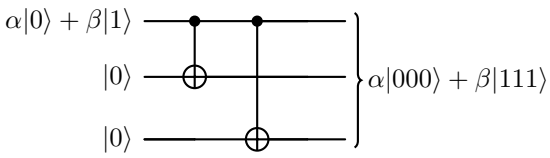


Figure 1.4. *Encoding circuit for the bit-flip code. Entanglement distributes the logical information without violating the no-cloning theorem.*

To solve the measurement problem, Shor introduced the concept of **syndrome extraction**. Instead of measuring each qubit (which would destroy the superposition $\alpha|000\rangle + \beta|111\rangle$), we compare pairs of these qubits to determine whether they are equal or different without gaining any information about their actual state.

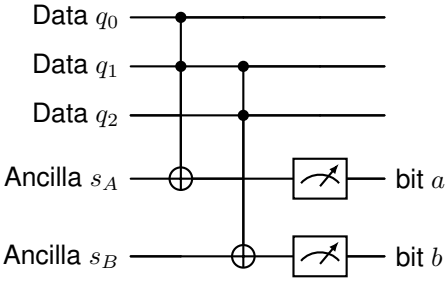


Figure 1.5. *Syndrome measurement: The ancilla qubits are activated when the two qubits that they monitor are different. Depending on which of the two ancilla qubits are activated, we can determine which of the qubits has flipped.*

As shown in Figure 1.5, measuring the ancilla qubits does not provide information about the state of each individual qubit, but it provides information about two qubits being different. The following table shows how the measuring of these ancilla qubits maps to specific identified errors.

Syndrome (s_A)	Syndrome (s_B)	Error Location	Correction
0	0	None	No action
1	0	q_0	Apply X_0
0	1	q_2	Apply X_2
1	1	q_1	Apply X_1

A Code to Detect Phase-flip Errors

The bit-flip code addresses X errors. The other fundamental quantum error is the phase-flip (Z), which changes the relative phase (the sign) in a superposition: $Z(\alpha|0\rangle + \beta|1\rangle) = \alpha|0\rangle - \beta|1\rangle$. This is strictly a quantum phenomenon and has no classical counterpart.

To correct a phase-flip, we can leverage a key property of quantum gates: a Hadamard gate (H) transforms a Z (phase-flip) error into an X error (bit-flip). Using the H gate, we can reuse the logic of the bit-flip code, where an X-parity check (i.e. comparing pairs of qubits) in the logical space effectively functions as a Z-check in the physical space.

Formally, the Hadamard gate maps the standard computational basis $\{|0\rangle, |1\rangle\}$ to the superposition basis $\{|+\rangle, |-\rangle\}$. In terms of operators, $HZH = X$.

The logical states are thus defined as:

$$|0\rangle_L = |++\rangle, \quad |1\rangle_L = |--\rangle$$

The encoding circuit entangles the qubits using the CNOT structure of the bit-flip code and then applies a global Hadamard transformation to switch the entire block into the relevant basis.

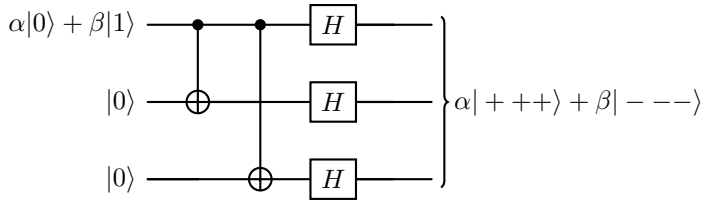


Figure 1.6. *Encoding circuit for the 3-qubit phase-flip code. The information is first distributed via entanglement, and then Hadamard gates transform the final state into the $\{|+\rangle, |-\rangle\}$ basis.*

Applying the same measuring technique to these three qubits in pairs, allows determining the syndrome, which in this case will point to a Z error. The Figure 1.7 shows the corresponding circuit.

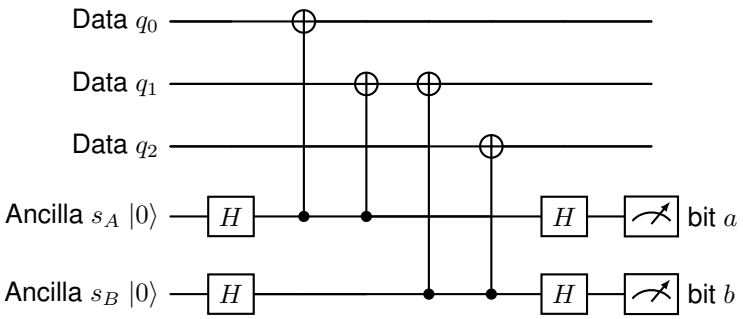


Figure 1.7. *Phase-flip syndrome measurement: The ancilla qubits are initialized in the $|+\rangle$ state using Hadamard gates. To perform the measurement in the X basis, CNOT gates are applied in reverse (ancilla as control, data as target) to exploit phase kickback. A final Hadamard gate returns the ancillas to the computational basis before measurement.*

The following table shows how each of the possible syndromes points to one specific error correction:

X_1X_2	X_2X_3	Error Location	Correction
0	0	None	No correction (I)
1	0	Qubit 1	Apply Z_1
1	1	Qubit 2	Apply Z_2
0	1	Qubit 3	Apply Z_3

The 9-qubit Shor Code: Correcting Arbitrary Errors

The 3-qubit code provides a solution to the first two challenges, but it only corrects one type of error (either bit-flip or phase-flip). To be able to correct flips in any of the three axis, Shor's proposed the **concatenation** of a bit-flip code and a phase-flip code. By nesting them, we create a 9-qubit block that protects against both bit-flips (X) and phase-flips (Z) simultaneously.

The logical basis states are constructed by taking three clusters of qubits. Each cluster is in a superposition ($|000\rangle \pm |111\rangle$) to handle phase errors, while each individual qubit within the cluster is tripled to handle bit-flip errors:

$$|0\rangle_L = \frac{(|000\rangle + |111\rangle)(|000\rangle + |111\rangle)(|000\rangle + |111\rangle)}{\sqrt{8}}$$

$$|1\rangle_L = \frac{(|000\rangle - |111\rangle)(|000\rangle - |111\rangle)(|000\rangle - |111\rangle)}{\sqrt{8}}$$

Z-type detectors are implemented with CNOT gates that have the ancilla as target and the data qubits as controls. X-type detectors are implemented using phase kickback: CNOT gates that have the ancilla as control and the data qubits as target.

To extract the syndrome of the 9-qubit Shor code, a total of 8 ancilla qubits are needed, 6 of them monitoring pairs of qubits to detect bit flips inside the clusters (Z-type detectors), and 2 of them monitoring full clusters to detect phase flips between two clusters (X-type detectors). The Table 1.1 shows the ancilla qubits and the data qubits they monitor.

Ancilla	Detector type	Targeted qubits	Error detected
a_1	Z-type	q_1, q_2	Bit-flip (X) in Block 1
a_2	Z-type	q_2, q_3	Bit-flip (X) in Block 1
a_3	Z-type	q_4, q_5	Bit-flip (X) in Block 2
a_4	Z-type	q_5, q_6	Bit-flip (X) in Block 2
a_5	Z-type	q_7, q_8	Bit-flip (X) in Block 3
a_6	Z-type	q_8, q_9	Bit-flip (X) in Block 3
a_7	X-type	$q_1, q_2, q_3, q_4, q_5, q_6$	Phase-flip (Z) in clusters 1 or 2
a_8	X-type	$q_4, q_5, q_6, q_7, q_8, q_9$	Phase-flip (Z) in clusters 2 or 3

Table 1.1. *Ancilla qubits to extract the syndrome in the 9-qubit Shor code.*

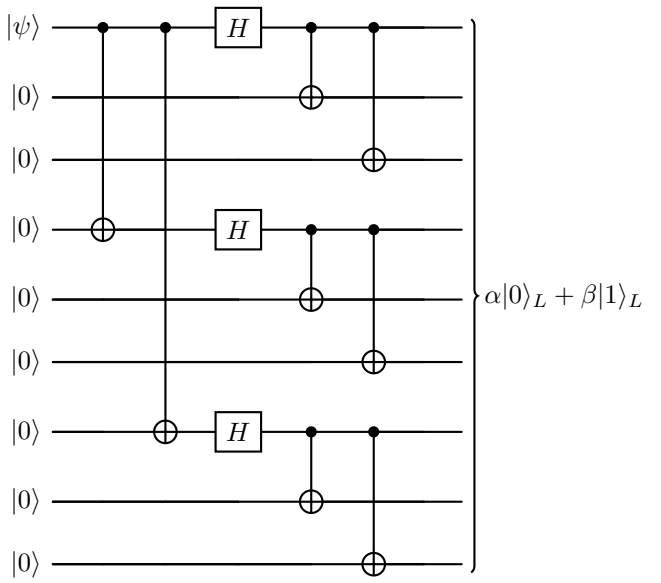


Figure 1.8. *Full encoding circuit for the 9-qubit Shor code. The initial CNOT and Hadamard gates encode the logical state into a 3-qubit phase-flip code. Subsequently, each of these three qubits is encoded into a 3-qubit bit-flip code, resulting in the final 9-qubit state.*

The Table 1.2 shows some examples of possible measured syndromes, the corresponding errors and the necessary recovery operation.

The Discretization of Quantum Errors

We have seen that the 9-qubit Shor code enables correcting both X and Z, but how can we correct an infinite number of possible small rotations?

The solution is surprisingly simple. It relies on the mathematics of quantum measurement. As proven by Knill and Laflamme [11], we do not actually need to correct any arbitrary error. Any arbitrary single-qubit error can be expressed as a linear combination of the Pauli matrices: I (identity, no error), X (bit-flip), Z (phase-flip) and Y (both bit and phase-flip).

By designing a quantum error correction code capable of detecting and correcting independent X and Z errors, such as the one designed by Shor, the act of measuring the error (the syndrome extraction) forces the continuous, arbitrary error to collapse into one of the discrete Pauli operators (I, X, Y and Z). Since I means no error and the Pauli Y operator is simply a combination of X and Z ($Y = iXZ$), **correcting X and Z is sufficient to correct any arbitrary error**. This phenomenon, known as the discretization of quantum errors, effectively reduces the continuity quantum error problem back to a discrete problem analogous to classical bit-flips.

It is important to note that phase-flip correction can be applied to any qubit of the cluster for which we detected the error. This is because the phase is encoded in the relative phase between the $|000\rangle$ and $|111\rangle$ components of the cluster, which changes by changing the phase of any of the three qubits. To correct the Y error, the combination of X and Z can be used.

Syndrome (Z-type) (a_1a_2, a_3a_4, a_5a_6)	**Syndrome (X-type)** (a_7, a_8)	**Error Type**	**Correction**
$(00, 00, 00)$	$(0, 0)$	None	I (No error)
$(10, 00, 00)$	$(0, 0)$	X_1	Apply X_1
$(11, 00, 00)$	$(0, 0)$	X_2	Apply X_2
$(01, 00, 00)$	$(0, 0)$	X_3	Apply X_3
$(00, 00, 00)$	$(1, 0)$	Z in cluster 1	Apply Z_1
$(00, 00, 00)$	$(1, 1)$	Z in cluster 2	Apply Z_4
$(00, 00, 00)$	$(0, 1)$	Z in cluster 3	Apply Z_7
$(10, 00, 00)$	$(1, 0)$	Y_1	Apply Y_1

Table 1.2. *Decoding the syndrome in the 9-qubit Shor code.*

Error Distinguishability

The fundamental requirement for a quantum error correction code is that errors that require different recovery operations must result in distinct syndromes. In formal terms, this means that the errors must map the logical state into orthogonal, distinguishable subspaces. If two physical errors produce the same syndrome but result in fundamentally different corrupted states, the decoder will not know which correction to apply, leading to a logical failure.

*Any qubit within a cluster suffering a phase-flip error in the 9-qubit Shor code results in the same syndrome. However, an operation on any of the three qubits of the cluster will fix the state of the logical qubit. This is known as degeneracy, and we say that the three phase-flip errors that can affect a cluster are **degenerated errors**.*

The 9-qubit Shor code works well precisely because it separates bit-flip and phase-flip errors into distinct, non-overlapping parity checks. For example, any bit-flip error in a single qubit will activate either one or two of the first 6 ancillas (see Table 1.2), which can map uniquely to the specific qubit that flipped. Similarly, any phase-flip error in a single qubit will activate one or two of the last 2 ancillas, which maps to the specific cluster that requires correction.

This ability to uniquely map syndromes to the appropriate recovery operation is what makes a QEC code successful. However, analyzing these subspaces manually becomes unsustainable for larger, more complex quantum codes. This scalability problem motivates the need for a robust, algebraic framework, leading directly to the Stabilizer Formalism that we discuss in the next section.

1.2.1 The Stabilizer Formalism

As quantum codes scale, representing the logical state as a vector of 2^n amplitudes becomes computationally intractable. Tracking the evolution of a highly entangled state qubit by qubit is unsustainable. The Stabilizer Formalism, introduced by Daniel Gottesman in 1997 [12], offers an elegant workaround: instead of describing the quantum state directly, we describe the set of operators that leave the state unchanged. More specifically, we focus on the operators that leave unchanged a valid state (i.e. a state where no error has occurred) but

modify an erroneous state.

Stabilizer Operators

The core concept of this framework relies on a simple mathematical condition. We say that an operator S **stabilizes** a quantum state $|\psi\rangle$ if applying S to the state does not alter it in any way:

$$S|\psi\rangle = |\psi\rangle \tag{1.1}$$

In linear algebra, an eigenvector $|\psi\rangle$ of an operator S satisfies $S|\psi\rangle = \lambda|\psi\rangle$, where λ is a constant called the eigenvalue. In the stabilizer formalism, we strictly look for states where $\lambda = +1$. Because the state remains unchanged, measuring S will yield a deterministic outcome of $+1$, indicating no error. It's important to note that measuring the $+1$ usually results in a 0 value for the classical syndrome bit.

In practice, a stabilizer operator is simply a combination of Pauli matrices (X, Y, Z) acting on a specific subset of qubits. For example, an operator Z_1Z_2 monitors the phase relationship of the first and second qubits. These operators are what we referred to as "syndrome detectors" or "parity checks" in the previous section when discussing the Shor code.

Note that an error affecting the qubits can also be represented as an operator (e.g. a bit-flip is a X Pauli matrix applied to the flipping qubit).

We can **measure** a stabilizer operator. Such measurement is implemented as a parity check on the data qubits that are connected to this operator, with the result being reflected in the state of an ancilla qubit. In the prior section, we saw how:

- Z-type stabilizers, which detect bit-flip errors, are measured with a CNOT gate that has the data qubits as source and the ancilla qubit as target.
- X-type stabilizers, which detect phase-flip errors, are measured with a CNOT gate that has the data qubits as target and the ancilla qubit as source, which produces a *phase kickback*.

We must also introduce the concept of **commutation**. In the mathematics of Pauli matrices, two operators either commute (meaning their order of application does not matter) or **anti-commute** (meaning swapping their order introduces a minus sign). Because both stabilizers and errors are Pauli operators, we can determine commutation properties for them.

Mathematically, two operators A and B commute if $AB = BA$, and anti-commute if $AB = -BA$. For Pauli matrices, identical types commute (e.g., an X error commutes with an X stabilizer), but different types acting on the same qubit anti-commute (e.g., an X error anti-commutes with a Z stabilizer).

The algebraic property of commutation is the core mechanism of error detection. If a quantum system is stabilized by an operator S, we can periodically measure S to check the health of the qubits connected to it:

- If no error has occurred, or if an error occurs that *commutes* with S, the state remains a valid $+1$ eigenstate. The measurement returns $+1$, telling the decoder that those specific qubits are safe (or not affected by an error detectable by S).
- If a physical error occurs that *anti-commutes* with S, the error flips the sign of the state. The measurement will return -1, acting as a localized alarm that an error has corrupted the monitored qubits.

We can see that if we can construct a set of operators that can detect all the possible errors affecting our physical qubits, we can determine the operations required to correct them.

The Stabilizer Group and Generators

*A **group** is a set equipped with an operation that combines any two elements to form a third element within the same set. The Pauli group is closed under matrix multiplication (the product of two Pauli operators is always a Pauli operator). A group whose elements commute between them is known as an **abelian group**.*

To build these stabilizer operators, we use the n-qubit Pauli group, denoted as $\mathcal{P}_n$. This group consists of all possible tensor products of the standard Pauli matrices (I, X, Y, Z) applied to n qubits, multiplied by a global phase factor $(\pm 1, \pm i)$.

Under this formalism, a quantum error correction code is defined by its **Stabilizer Group** $(\mathcal{S})$, which is a commuting subgroup of $\mathcal{P}_n$, that is, a subset of $\mathcal{P}_n$ for which all its members commute between them.

The reason why this is a strict requirement is that, for a QEC code to be valid, all the operators in the group must share a common set of eigenvectors with eigenvalue $+1$. These eigenvectors are the logical states of the QEC code, and are not affected by any of the stabilizer operators. Only commuting operators can share a common set of eigenvectors.

Additionally, for $\mathcal{S}$ to define a valid code, the operator $-I$ must not be an element of the group. If $-I$ was an element of $\mathcal{S}$, then we would have $-I|\psi\rangle = |\psi\rangle$. This only holds if $|\psi\rangle = 0$, meaning the code would have no valid quantum states.

We've defined the $\mathcal{S}$ as a set of commuting operators. However, we do not need every single operator in $\mathcal{S}$ to define the code. We only need a small, independent set of operators called **generators**. Just as a three basis vectors can define an entire 3D space, multiplying these generators together produces all the other stabilizers in the group. This set of independent operators represents the set of detectors that we will have to implement in the quantum circuit.

For an n-qubit system, choosing $n - k$ independent commuting generators uniquely defines a QEC code capable of encoding k qubits. Such generators create a protected logical subspace with 2^k logical states.

Standard [[n, k, d]] Notation

Note that the parameter n does not include the number of ancilla qubits required to extract the syndrome, as it depends on the actual implementation.

In the field of Quantum Error Correction, stabilizer codes are conventionally described using the $[[n, k, d]]$ notation, which provides a summary of the code's cost and capacity:

- n **(Physical Qubits):** The total number of physical qubits used to construct the code.
- k **(Logical Qubits):** The number of logical qubits encoded.
- d **(Code Distance):** The minimum number of physical errors required to cause an undetected logical failure.

Under this convention, the Shor code is formally classified as a $[[9, 1, 3]]$ code, meaning it uses nine physical qubits to protect one logical qubit with a distance of three, and requires eight $(n - k)$ stabilizers.

The distance directly determines the error-handling capabilities of the code through two fundamental rules:

1. **Detection:** A code can detect any error affecting up to $d-1$ physical qubits.
2. **Correction:** A code can correct up to t physical errors:

$$t = \left\lfloor \frac{d-1}{2} \right\rfloor \tag{1.2}$$

For the Shor code ($d = 3$), these rules imply that the code can detect up to 2 errors and correct $t = 1$ error. If two errors occur simultaneously, the syndrome would cause a wrong correction. This is a permanent trade-off in QEC: increasing the correction threshold t, requires increasing the distance d, which requires more physical qubits n.

The Threshold Theorem

Proven by Aharonov and Ben-Or [6], the Threshold Theorem gives us a framework for arbitrarily long quantum computations.

We have seen so far that through QEC we can effectively lower down the logical error rate (i.e. the error introduced by the overhead QEC in the circuit is lower than the error effectively corrected, so the resulting logical error is below the physical error rate).

We have also seen how a fundamental parameter of a code is the distance, d. It determines the ability of the code to correct simultaneous errors and, therefore, it relates to the capacity of the code to reduce the error rate. And we know that the distance of a code is correlated with the code overhead: to increase the distance, we need to add more physical qubits, n, to codify the same number of logical qubits, k.

However, as we increase the complexity of quantum computations, we need quantum circuits that are wider (more qubits) and deeper (more steps in the calculation). Because the probability of error is constant through the execution, during the execution of an arbitrarily deep circuit, a non-recoverable error will eventually happen.

The Threshold Theorem combines all this to provide a foundational proof of the usefulness of Quantum Error Correction. It proves that when we increase the distance of a code, hence introducing overhead but improving the code capacity, the logical noise rate that we obtain behaves differently depending on the physical error rate that our hardware has:

- If the physical error rate, p is below a certain value, p_{th}, which depends on the code itself, increasing the distance reduces the logical noise. That is, the benefit of increasing the code capacity is greater than the drawback of increasing the circuit overhead.
- If p is above p_{th}, increasing the distance increases the logical noise. That is, the benefit of increasing the code capacity is smaller than the drawback of increasing the circuit overhead.

Note that the threshold, p_{th}, of a given code is not an isolated property of the code. It depends on the actual implementation of the code in a certain hardware. In Chapter 2 we will show how mapping a given code to a given system (a set of qubits with a given interconnectivity) adds overhead that change the Threshold of the result.

The value at which the behaviour change is what we call the Threshold, p_{th}, of a QEC code. We can conclude that, if we have hardware whose physical error rate p is below the threshold p_{th} of a certain code, we can arbitrarily increase the distance of such code to reduce the effective noise. This compensates the increase in the chances of a non-recoverable error (when a circuit depth increases) by increasing the distance of the code.

In Chapter 5, we will simulate various QEC codes to visualize this phase transition. By plotting the logical error rate p_L as a function of the physical error rate p for different distances, we will numerically identify the threshold p_{th} as the precise crossing point where scaling the distance inverts its effect on logical noise.

CHAPTER 2

State of the art of QEC codes: Surface codes, color codes and QLDPC codes

Nothing yet

Papers en donde se ve como mapean los códigos 6.6.6 y 4.8.8 a un procesador (un grid cuadrado)

4.8.8: (see FIG 4) https://arxiv.org/pdf/0806.4827 [13]

6.6.6: (see FIG 1a) https://arxiv.org/html/2412.14256v1 [14]

Paper que estudia justo códigos de color y sus umbrales: https://arxiv.org/pdf/1108.5738 [15]

CHAPTER 3

QEC decoders for color codes

Nothing yet

3.1 MWPM

3.2 Union-find

3.3 BP+OSD

3.4 Projection decoders (Delfosse, 2014)

3.5 Restriction decoders (Kubica & Delfosse, 2023)

3.6 Möbius decoder (Sahay & Brown, PRX Quantum, 2022)

3.7 Concatenated MWPM decoder (Lee, Li & Bartlett, Quantum, 2025)

3.8 Correlated matching decoder (Liu, Wu & Lao, 2025)

CHAPTER 4

Modeling quantum error

CHAPTER 5

Implementation and simulation of QEC codes: Framework setup and baseline models

We have so far conducted a profound theoretical study of the different QEC codes, decoders, and error models. Moving from theory to practice requires specialized computational tools capable of simulating quantum systems efficiently.

Simulating arbitrary quantum circuits is exponentially hard for classical computers; however, the Gottesman-Knill theorem [16] guarantees that circuits restricted to Clifford group operations (which include stabilizer measurements, CNOTs, and Pauli gates) can be simulated efficiently in polynomial time.

The Gottesman-Knill theorem is referred to in the original Gottesman paper simply as Knill's theorem, as it was initially only privately communicated. It is now commonly known as the Gottesman-Knill theorem to recognize both Knill's original insight and the subsequent formal proof by Gottesman using his Stabilizer formalism.

In this chapter, we establish our experimental framework using `stim` [9], a high-performance Clifford simulator that relies on the referred Gottesman-Knill theorem and its subsequent improvements by Aaronson and Gottesman[17], and incorporates some additional improvements. We will build our simulations progressively: starting with a basic repetition code to understand the mechanics of syndrome extraction and decoding, moving through different noise models, and finally implementing the 9-qubit Shor code to empirically demonstrate the Threshold Theorem.

Class Note

[Source code and repository structure] The code required to execute the simulations presented in this work is available in a git repository at https://github.com/othermore/TFM_QEC_2D_color_code_decoders.
The code is located in the `src` directory. Each chapter has its own directory and python notebook. The specific code for each simulation is located in separate files (starting with `s` and the number of the simulation) and included in the notebook.
Libraries and resources required for multiple simulations are located in the directory `src/lib`.

5.1 Simulating QEC codes and decoders with `stim`

Stim is a QEC specialized simulator. The primary advantage of `stim` is its ability to sample detection events –syndromes– and logical observables –flips in the logical state of our code– extremely fast. Instead of tracking the full state vector, it tracks how Pauli errors propagate through the circuit. To utilize this tool, a QEC simulation requires three main components: a circuit defining the code structure, a noise model that injects errors, and a classical decoder that interprets the syndromes.

5.1.1 Simulating the repetition code under code-capacity noise

To introduce the simulation pipeline, we begin with the simplest QEC protocol: the distance-d repetition code. As discussed in Chapter 1, this code protects against a single type of error (e.g., bit-flips) by encoding one logical qubit into d physical data qubits, requiring $d-1$ additional ancilla qubits to perform parity measurements.

The depolarizing channel is a standard symmetric noise model where a qubit has a probability p of suffering an error, with the error being equally likely to be the Pauli X, Y, or Z operator ($p/3$ each). This contrasts with asymmetric noise models, such as pure phase-damping or amplitude-damping channels.

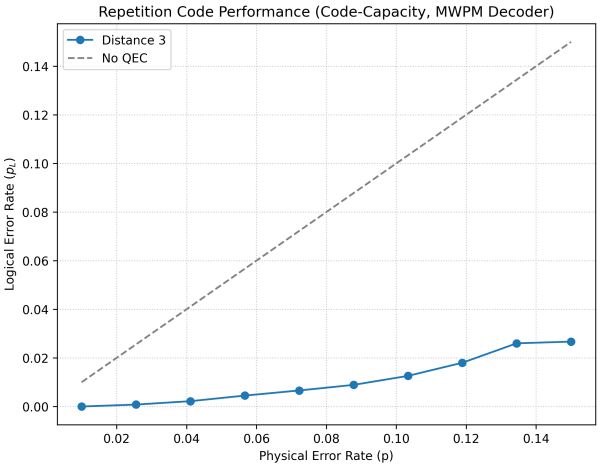
For our baseline simulation, we use a **code-capacity noise model**. As seen in Chapter 4, this is a highly idealized model where perfect, noiseless gates are used for encoding and syndrome extraction. In this model, errors are only artificially injected into the data qubits (typically via a depolarizing channel) to simulate decoherence over time.

Stim makes it exceptionally easy to implement a basic repetition code. The building blocks of this simulation are:

- **The QEC code circuit**. `stim`'s library already provides a method to easily generate a basic repetition code. This generates not only the circuit for the encoding, but the necessary ancilla qubits and measurements for syndrome extraction.
- **Injecting the error**. As part of building the circuit, `stim` allows inputing the physical error rates for different types of noise. In

this case, to simulate code-capacity noise, we use the parameter `before_round_data_depolarization`.

- **Decoding the error**. The `pymatching` library will allow us to decode the error using the MWPM algorithm that we discussed in Chapter 3. For such decoding to happen, `stim` requires the circuit to have specific tags appended (`DETECTORS`) to specific measures. In this example, the generated circuit already contains them.



The x=y dotted line helps comparing the QEC protocol to not implementing any QEC. If the logical error rate is above the physical error rate, the QEC protocol is adding errors.

Figure 5.1. *Simulation 1: Logical error for a basic repetition code using the MWPM decoder*

The code in Listing 5.1 illustrates how this simulation can be performed. It is important to understand how `stim` works. The `sample` method returns two things:

1. A binary array of detection events (in the code, `syndromes`). It contains one value for each measured detector. The value is 0 if the detector observed the expected parity, and 1 if the detector was activated, indicating the presence of an error.
2. A binary array of logical outcomes (in the code, `actual_observables`). Rather than representing logical states, these values act as boolean flags. A value of 1 indicates that the accumulated physical errors during that shot successfully flipped the corresponding logical observable.

During the simulation, errors can occur with a certain probability p. When they happen, they are tracked and cascade down to the detectors and observables, allowing `stim` to determine both if a detector is triggered and if an observable flips. If the code and the decoding succeed, both should match. Because flips, rather than states, are tracked, the circuit does not have to be prepared on a certain state, we only evaluate its capacity to preserve its state.

The preparation of the initial state of a circuit is not generally required. However, noise models can be asymmetric (e.g. a qubit may be more likely to flip from $|1\rangle$ to $|0\rangle$ than from $|0\rangle$ to $|1\rangle$) and so can be QEC codes (the 9-qubit Shor code has better protection against bit-flip errors than against phase-flip errors). In these cases, preparing the initial state may be required.

Figure 5.1 shows the resulting plot, displaying the logical error rate as a function of the physical error rate. It is worth noting that, as a basic repetition code, this

code only corrects for bit-flip errors.

```
import stim
import pymatching
import numpy as np
import matplotlib.pyplot as plt

def sample_repetition_code_mwpm(distance: int = 3, shots: int =
    10000) -> None:
    """
␣␣␣␣Simulates␣a␣repetition␣code␣over␣a␣range␣of␣physical␣error␣
    rates,
␣␣␣␣using␣the␣pymatching␣MWPM␣decoder.
␣␣␣␣"""
    physical_error_rates = np.linspace(0.01, 0.15, 10)
    logical_error_rates = []

    for p in physical_error_rates:
        # Define the circuit
        circuit = stim.Circuit.generated(
            "repetition_code:memory",
            distance=distance,
            rounds=1,
            before_round_data_depolarization=p
        )

        # Sample the circuit to get syndromes and actual
            observables
        sampler = circuit.compile_detector_sampler()
        syndromes, actual_observables = sampler.sample(shots,
            separate_observables=True)

        # Decode the syndromes using PyMatching
        matcher = pymatching.Matching.from_stim_circuit(circuit)
        predicted_observables = matcher.decode_batch(syndromes)

        # Calculate the logical error rate as the cases where the
        # actual observables differ from the predicted ones
        num_errors = np.sum(predicted_observables !=
            actual_observables)
        logical_error_rates.append(num_errors / shots)

    return physical_error_rates, logical_error_rates


def plot_repetition_code_mwpm_performance(distance: int = 3, shots
    : int = 10000) -> None:
    """
␣␣␣␣Simulates␣a␣repetition␣code␣over␣a␣range␣of␣physical␣error␣
    rates,
␣␣␣␣decodes␣using␣PyMatching,␣and␣plots␣the␣logical␣error␣rate.
␣␣␣␣"""
    physical_error_rates, logical_error_rates =
        sample_repetition_code_mwpm(distance, shots)
    # Plotting the results
    plt.figure(figsize=(8, 6))
    plt.plot(physical_error_rates, logical_error_rates, marker='o'
        , label=f'Distance␣{distance}')

    # Reference line to show the break-even point where QEC stops
```

```
        helping
    plt.plot(physical_error_rates, physical_error_rates, linestyle
        ='--', color='gray', label='No QEC')
    plt.title('Repetition Code Performance (Code-Capacity, MWPM
        Decoder)')
    plt.xlabel('Physical Error Rate (p)')
    plt.ylabel('Logical Error Rate ($p_L$)')
    plt.legend()
    plt.grid(True, linestyle=':', alpha=0.7)

# Execute to input the main parameters
if __name__ == "__main__":
    plot_repetition_code_mwpm_performance(distance=3, shots=10000)
```

Listing 5.1. *Simulation 1: Basic repetition code simulation*

5.1.2 Decoders: From Minimum Weight Perfect Matching to Custom Logic

Once the circuit is executed and noise is applied, `stim` outputs an array of detection events. The role of the decoder is to map these events to a predicted logical error.

As seen in Chapter 3, the standard approach for 1D repetition codes and 2D surface codes is the **Minimum Weight Perfect Matching (MWPM)** algorithm. In our framework, we implement this using the `PyMatching` library.

To better understand the internal workings of a decoder, we implement a custom **Majority Vote Decoder**. Unlike the graph-based approach of MWPM, this decoder resolves errors by directly evaluating the local parity checks and "voting" on the most likely state of the data qubits.

The code Listing 5.2 shows a very simple implementation of such decoder, which can be used with the same repetition circuit created in Listing 5.1.

```
import numpy as np
import matplotlib.pyplot as plt

def decode_single_shot(shot_syndrome, num_observables,
    detectors_per_observable, distance):
    """
    Decodes a single 1D array of detectors using majority vote.
    """
    shot_predictions = []

    # We iterate over each observable, which corresponds to a
        block of detectors
    # This would allow codes encoding multiple logical qubits
    for i in range(num_observables):
        # The number of detectors in our code, for a given
            observable
        # is a function of the code distance.
        # We may receive additional detectors (frontier detectors)
            that we can ignore.
        # For the 3-qubit repetition code, we have 2 detectors per
            observable.
```

```
        # We need to iterate over the blocks of detectors for each observable
        start_index = i * detectors_per_observable
        end_index = start_index + (distance -1)
        local_syndrome = shot_syndrome[start_index:end_index]

        # For a repetition code, decoders are based on pairs of data qubits
        # that are chained, e.g. (q0, q1) and (q1, q2).
        # We achor to the first qubit, assume that it has not flipped (set to 0).
        qubits_state = [0]
        current_state = 0

        # Let's iterate over the detectors
        for detector_value in local_syndrome:
            # If a given detector is triggered, it indicates a flip
            # of this qubit relative to the previous one.
            if detector_value == 1:
                # We flip with respect to the prior qubit state
                current_state = 1 - current_state
            # Store the state of this qubit
            qubits_state.append(current_state)

        # If, as we assumed, q0 has not flipped, then our qubit_state tells us
        # which qubtis have flipped.
        # But if q1 has flipped, then our qubit_state is the opposite
        # of the actual flip-state of each qubit.

        # If we have more 1s than 0s, that is a very unlikely scenario
        # (2 bits flipped)so we can infer that q1 had actually flipped
        # (and our qubits_state is the opposite of the actual flip-state).
        num_ones = sum(qubits_state)
        num_zeros = len(qubits_state) - num_ones

        # We now need to make our flip-prediction for the observable.
        # Stim's circuit will contain the observable measurement in the last qubit
        # of the block, because it uses the last measurement.
        if num_ones > num_zeros:
            # If we have more 1s than 0s, we assume that our flip-states are inverted,
            # so we take the opposite of the last qubit's flip-state
            # as our prediction for the observable.
            local_prediction = 1 - qubits_state[-1]
        else:
            # If we have more 0s than 1s, we assume our flip-states are correct.
            # So the flip-state of the last qubit is the right flip-state
            # for the observable.
```

```
            local_prediction = qubits_state[-1]

        shot_predictions.append(local_prediction)

    return shot_predictions

def decode_batch_majority_vote(syndromes, circuit, distance):
    """
    Decodes a 2D matrix of syndromes by applying the single-shot decoder to each row.
    """
    model = circuit.detector_error_model()
    num_observables = model.num_observables
    detectors_per_observable = model.num_detectors // num_observables

    batch_size = syndromes.shape[0]
    all_predictions = []

    for shot_idx in range(batch_size):
        # Get one shot's syndrome
        shot_syndrome = syndromes[shot_idx]

        # Decode this shot
        shot_predictions = decode_single_shot(
            shot_syndrome,
            num_observables,
            detectors_per_observable,
            distance
        )

        all_predictions.append(shot_predictions)

    return np.array(all_predictions)
```

Listing 5.2. *Simulation 2: Implementation of a simple Majority Vote decoder for a repetition code*

As seen in Figure 5.2, for simple repetition codes under basic noise models, both decoders perform similarly, effectively reducing physical errors. In the following section we will stress-test our decoders to see how their efficiency decays with different noise models.

5.1.3 Introducing circuit-level noise

While the code-capacity model is useful for validating decoders, it does not reflect physical reality. In a real quantum computer, the gates used to entangle the qubits and the measurements used to extract the syndrome are themselves noisy.

To illustrate this, we expand our simulation to include **circuit-level noise** and **phenomenological noise**. As seen in Chapter 4, in this model, we inject depolarizing noise into single-qubit and two-qubit gates, and we add a probability of measurement readout errors. For the purpose of this test, we will use a very simple noise model, where the three types of noise added to the

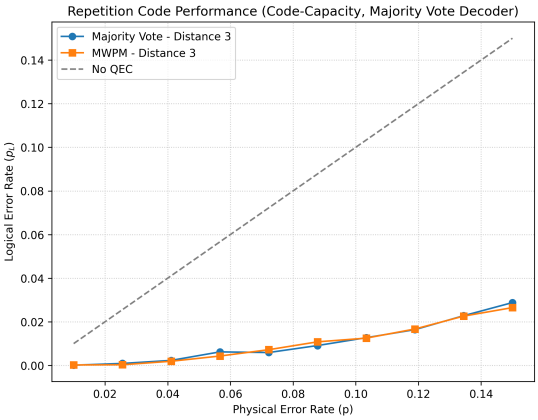


Figure 5.2. *Simulation 3: Comparison of logical error for a basic repetition code using Majority Vote and MWPM decoders*

circuit have the same probability (ratio 1:1:1). Figure 5.3 shows the result of the simulation. This is easily implemented with `stim` in the circuit creation, as seen in Listing 5.3.

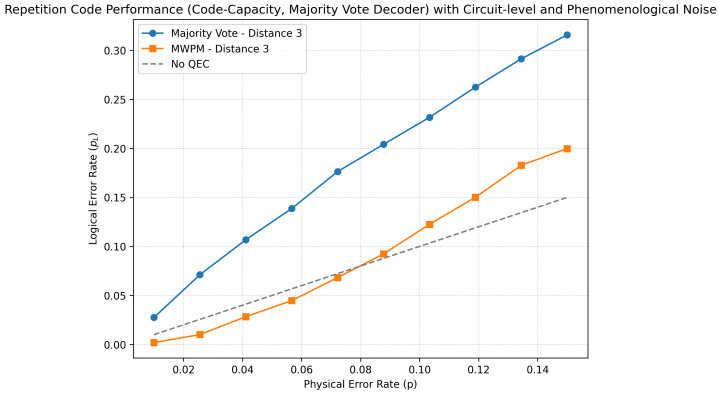


Figure 5.3. *Simulation 4: Impact of circuit-level and phenomenological noise on the repetition code performance. Introducing more complex noise significantly degrades the logical error suppression compared to the idealized code-capacity baseline. With this additional noise, MWPM shows its advantage over the Majority Vote because of its capacity to consider the probabilities of each error to select the most probable set of errors that could have caused the observed syndrome.*

```
circuit = stim.Circuit.generated(
    "repetition_code:memory",
    distance=distance,
    rounds=1,
    before_round_data_depolarization=p,
    after_clifford_depolarization=p,
    before_measure_flip_probability=p
```

```
)
```

Listing 5.3. *Generation of a repetition code circuit using Stim.*

As shown in Figure 5.3, realistic noise drastically reduces the effectiveness of the code, but it also shows the importance of the decoder. Handling circuit-level noise requires complex spatial-temporal decoding graphs, which makes MWPM a better solution than a basic Majority Vote decoder.

At this point, we can also study the effect of distance increases in the code. Figure 5.4 shows how increasing the distance improves the results of the MWPM decoder, but it degrades the result of the Majority Vote decoder. This means that, for this specific noise model, MWPM is operating under the threshold, so the error correcting capacity of the protocol can be increased by increasing the distance. However, our Majority Vote decoder operates above the threshold.

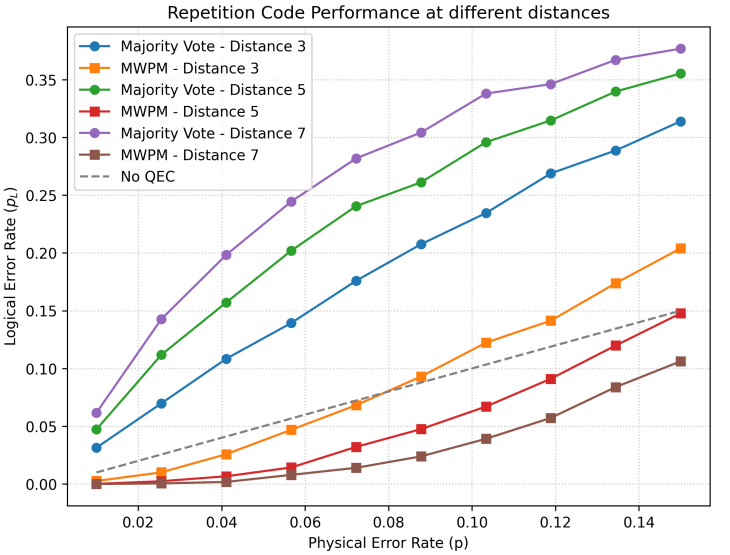


Figure 5.4. *Simulation 5: Under a complex noise model (code-capacity, circuit-level and phenomenological) increasing the distance of the code has the opposite effect for each of the decoders.*

To maintain focus on the architectural advantages of QEC codes (and, more specifically, 2D color codes) and their fundamental decoding algorithms, the remainder of this thesis will operate primarily under the code-capacity model, unless stated otherwise.

CHAPTER 6

Simulating 2D triangular color codes with a 4.8.8 lattice geometry and a code-capacity noise model

CHAPTER 7

Comparative analysis and discussion

CHAPTER 8

Evaluating potential improvements to projection decoding for 2D color codes

Nota: para metodología de la investigación hay que buscar una revista en la que tuviese sentido publicar tu TFM.

Esta revista que está bastante especializada y aceptan artículos tipo tutorial, en lo que podría encajar: https://tqe.ieee.org/special-section-on-quantum-engineering-education/

Otras opciones serían: PRX Quantum (Sección “Tutorials”)

Bibliography

[1] Michael A Nielsen and Isaac L Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 2010.

[2] Richard P Feynman. Simulating physics with computers. *International Journal of Theoretical Physics*, 21(6/7), 1982.

[3] Peter W Shor. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th Annual Symposium on Foundations of Computer Science*, pages 124–134. IEEE, 1994.

[4] Peter W. Shor. Scheme for reducing decoherence in quantum computer memory. *Physical Review A*, 52(4):R2493–R2496, 1995.

[5] Andrew M. Steane. Error correcting codes in quantum theory. *Physical Review Letters*, 77(5):793–797, 1996.

[6] Dorit Aharonov and Michael Ben-Or. Fault-tolerant quantum computation with constant error. In *Proceedings of the twenty-ninth annual ACM symposium on Theory of computing*, pages 176–188, 1997.

[7] A Yu Kitaev. Fault-tolerant quantum computation by anyons. *Annals of Physics*, 303(1):2–30, 2003.

[8] H. Bombin and M. A. Martin-Delgado. Topological quantum distillation. *Physical Review Letters*, 97(18), October 2006.

[9] Craig Gidney. Stim: a fast stabilizer circuit simulator. *Quantum*, 5:497, 2021.

[10] William K. Wootters and Wojciech H. Zurek. A single quantum cannot be cloned. *Nature*, 299(5886):802–803, 1982.

[11] Emanuel Knill and Raymond Laflamme. Theory of quantum error-correcting codes. *Physical Review A*, 55(2):900, 1997.

[12] Daniel Gottesman. *Stabilizer codes and quantum error correction*. PhD thesis, California Institute of Technology, 1997. arXiv preprint quant-ph/9705052.

[13] Austin G. Fowler. Two-dimensional color-code quantum computation. *Physical Review A*, 83(4), April 2011.

[14] N. Lacroix et al. Scaling and logic in the colour code on a superconducting quantum processor. *Nature*, 645(8081):614–619, May 2025.

[15] Andrew J Landahl, Jonas T Anderson, and Patrick R Rice. Fault-tolerant quantum computing with color codes. *arXiv preprint arXiv:1108.5738*, 2011.

[16] Daniel Gottesman. The heisenberg representation of quantum computers. In *Group22: Proceedings of the XXII International Colloquium on Group Theoretical Methods in Physics*, pages 32–43. International Press, 1999. arXiv preprint quant-ph/9807006.

[17] Scott Aaronson and Daniel Gottesman. Improved simulation of stabilizer circuits. *Physical Review A*, 70(5):052328, 2004.